

Time Series Analysis

for Intelligent Multi-Sensor Fusion

PHASE 4 — RESEARCH-GRADE NOTES

Trinetra-AI

An Intelligent Multi-Sensor Fusion Framework for GPS-Denied Navigation

Every concept here is developed for the sensors of Trinetra-AI — **accelerometer, gyroscope, magnetometer, IMU, GPS, barometer**, and the future **quantum magnetometer**. The recurring question is not “what is this time-series idea?” but “*where does it live inside the Trinetra-AI pipeline, and how does it help bound drift in GPS-denied navigation?*” Intuition first, then formal definitions, then derivations, then code, then the exact project mapping.

Beginner intuition → formal definitions → derivations → code → project mapping.

Contents

1	Introduction	2
1.1	What is a time series?	2
1.2	Why sensor data is inherently sequential	2
1.3	Why time series is critical in robotics	2
1.4	Why navigation systems depend on time series	2
1.5	Time series across the mission domains	3
2	Time Series Fundamentals	3
2.1	Temporal structure	3
2.2	Signal components	4
2.3	Stationarity	4
2.4	Consolidated reference	5
3	Sensor Time Series	5
3.1	The universal sensor error model	5
3.2	Per-sensor summary	6
3.3	Reading the signatures	6
3.4	Preprocessing checklist	6
4	Sliding Window	7
4.1	Formal definition	7
4.2	Why windows, and how the models use them	7
5	Time Series Forecasting	8
5.1	One-step, multi-step, recursive, direct	8
5.2	What Trinetra-AI can forecast — and why	9
6	Sequence Modeling	9
6.1	Vocabulary	9
6.2	RNN — the baseline recurrence	9
6.3	LSTM — gated long memory	10
6.4	GRU — the lean cousin	10
6.5	TCN — dilated causal convolutions	10
6.6	Transformer, Informer, Time-Series Transformer	11
6.7	Model comparison	11
7	Drift Detection	11
7.1	Sensor drift (physical)	11

7.2	Concept / data / model drift (ML)	12
7.3	Detection & prediction methods	12
8	Temporal Feature Engineering	13
8.1	Feature catalogue	13
8.2	Which features for which module	13
9	Time Series Datasets	14
10	Libraries	15
11	Research Papers	15
11.1	Foundational sequence models	16
11.2	Learned inertial navigation (most relevant)	16
11.3	Magnetic & generative (optional)	16
12	Project Integration	16
13	Interview Preparation	17
13.1	30 interview questions	17
13.2	20 research questions	18
13.3	10 viva questions	19
14	Practical Implementation	19
15	Summary, Glossary & Roadmap	20
15.1	Summary	20
15.2	Glossary	20
15.3	Roadmap: how Phase 4 connects to Phase 5 (Signal Processing)	20

1 Introduction

1.1 What is a time series?

A **time series** is a sequence of observations indexed by time,

$$X = \{x_{t_1}, x_{t_2}, \dots, x_{t_N}\}, \quad t_1 < t_2 < \dots < t_N,$$

where each x_{t_i} may be a scalar (barometric pressure) or a vector (a 3-axis accelerometer sample $\mathbf{x}_t = [a_x, a_y, a_z]^T$). The defining property is that **order carries information**: permuting the samples destroys it.

That single property separates time-series analysis from ordinary tabular machine learning, where rows are assumed exchangeable. *Intuition*: a photograph tells you a state; a video tells you a *story*. Sensor data is a video — the meaning of “the accelerometer reads 9.8 m/s²” depends entirely on what came just before it.

1.2 Why sensor data is inherently sequential

A navigation state evolves by physics, $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$: position is the integral of velocity, velocity the integral of acceleration. Because each state depends on the previous one, the stream that *observes* that state is sequential by construction. You literally cannot compute displacement from one accelerometer sample — you must integrate a *sequence*.

1.3 Why time series is critical in robotics

A robot closes a **perception** → **estimation** → **control** loop tens to hundreds of times per second. Every block consumes time series: raw streams in, filtered state estimates out, commands as a function of the recent past. Latency, sample-rate mismatch, and temporal misalignment are first-class engineering concerns, not afterthoughts.

1.4 Why navigation systems depend on time series

Strapdown inertial navigation integrates the IMU:

$$\mathbf{v}_t = \mathbf{v}_0 + \int_0^t (\mathbf{R}(\tau) \mathbf{a}^{\text{body}}(\tau) - \mathbf{g}) d\tau, \quad \mathbf{p}_t = \mathbf{p}_0 + \int_0^t \mathbf{v}(\tau) d\tau.$$

A constant accelerometer bias b therefore produces a position error $\Delta p(t) = \frac{1}{2}bt^2$ — *quadratic* in time. This is why small errors compound and why the temporal structure of error is the entire game in GPS-denied navigation.

1.5 Time series across the mission domains

Domain	Temporal challenge	Trinetra-AI relevance
GPS-denied nav	no absolute fix; unbounded drift over time	the core problem this project targets
Sensor fusion	recursively combine time-aligned streams	predict/update is inherently temporal
Quantum navigation	cleaner field stream, new noise physics	same windowing/fusion, better inputs
Autonomous vehicles	GNSS multipath, high dynamics	bridge short outages; predict drift
UAVs / drones	vibration, fast rotation, GPS loss in-doors	high-rate gyro handling; drift-aware fusion
Space navigation	no GPS beyond MEO; long integration	extreme drift discipline; magnetic/star aiding

Time series is not one phase among many — it is the *substrate* the whole project runs on. Classical filters, learned modules, and evaluation all operate on windows of temporally-ordered sensor data. The dominant enemy, **unbounded drift under GPS-denial**, is precisely a temporal, forecastable phenomenon — which is why this phase is foundational.

2 Time Series Fundamentals

We group the fundamentals into three clusters — *temporal structure*, *signal components*, and *stationarity* — with definitions, mathematics, sensor meaning, and code. A consolidated reference table closes the section.

2.1 Temporal structure

The **time index** is the position t_i of a sample in the sequence; a **timestamp** is its absolute clock value. The **sampling frequency** f_s (Hz) is samples per second and the **sampling interval** (period) is $T_s = 1/f_s$; **time resolution** is the smallest distinguishable time difference, bounded by T_s .

Temporal dependency is the degree to which x_t depends on its past. It is quantified by the **autocovariance** $\gamma(k) = \text{Cov}(x_t, x_{t-k})$ and the normalised **autocorrelation function (ACF)**

$$\rho(k) = \frac{\gamma(k)}{\gamma(0)} = \frac{\mathbb{E}[(x_t - \mu)(x_{t-k} - \mu)]}{\sigma^2} \in [-1, 1].$$

Physical sensor signals are strongly autocorrelated at short lags (physics is smooth); white noise has $\rho(k) = 0$ for $k \neq 0$.

Time synchronization. Different sensors run on different clocks at different rates; aligning them to one timeline (interpolation, hardware timestamping, offset estimation) is a prerequisite for fusion, and is *the* most common silent-failure mode of a fusion stack. **Nyquist reminder (bridge to Phase 5):** to represent a component at frequency f you need $f_s > 2f$, else it aliases and can masquerade as drift.

2.2 Signal components

Classical decomposition writes a series as an additive (or multiplicative) sum of components:

$$x_t = \underbrace{T_t}_{\text{trend}} + \underbrace{S_t}_{\text{seasonal}} + \underbrace{C_t}_{\text{cyclic}} + \underbrace{\varepsilon_t}_{\text{noise}}.$$

Trend T_t is slow monotone change; **seasonality** S_t a *fixed-period* pattern; **cyclic** C_t repetition *without* a fixed period; **noise** ε_t the irreducible random part.

Component	Sensor meaning	Consequence
Trend T_t	gyroscope bias drift; baro pressure falling on climb	integrates into growing error
Seasonality S_t	magnetic distortion repeating each robot loop; thermal duty cycle	exploitable / removable pattern
Cyclic C_t	gait cycles in pedestrian IMU (period varies with speed)	feature for step detection
Noise ε_t	white / random-walk sensor noise	sets fusion covariance R
Outliers	GPS multipath jump; magnetometer spike near a motor	must be gated before fusion
Missing values	dropped packets, GPS loss	triggers GPS-denied mode; impute/-mask

2.3 Stationarity

A series is **strictly stationary** if its full joint distribution is time-invariant. It is **weakly (covariance) stationary** if only the first two moments are constant:

$$\mathbb{E}[x_t] = \mu, \quad \text{Var}(x_t) = \sigma^2, \quad \text{Cov}(x_t, x_{t+k}) = \gamma(k) \quad \forall t.$$

Otherwise it is **non-stationary**.

Raw navigation signals are almost always non-stationary (a moving vehicle's statistics change constantly). Two consequences: (1) classical models such as ARIMA assume stationarity and require *differencing*/detrending first; (2) *drift itself is a non-stationarity*, so detecting drift (§7) is detecting a change in the signal's statistics.

```
from statsmodels.tsa.stattools import adfuller, acf
import numpy as np
# Augmented Dickey-Fuller: null = has a unit root (non-stationary)
def stationary(x, alpha=0.05):
    stat, p, *_ = adfuller(np.asarray(x, float))
    return p < alpha # True => reject null => stationary-ish
# Autocorrelation of a gyro axis, first 30 lags:
rho = acf(gyro_z, nlags=30, fft=True)
```

2.4 Consolidated reference

Concept	Math	Sensor example	Role in Trinetra-AI
Sampling freq.	$T_s = 1/f_s$	IMU 200 Hz, GPS 1 Hz	window sizing & fusion cadence
Time sync	$t_i^A \leftrightarrow t_j^B$	IMU vs GPS clocks	misalignment = biased fusion
Autocorrelation	$\rho(k)$	smooth accel vs white noise	feature for health/drift
Trend	T_t	gyro bias drift	target of drift prediction
Seasonality	S_t	loop-closure magnetic pattern	map-matching cue
Outlier	robust z	GPS multipath jump	anomaly/fault detection
Missing	mask/impute	GPS dropout	triggers GPS-denied mode
Stationarity	ADF test	static vs moving IMU	model choice; drift = non-stationarity

Interview prompts. *Why does a constant accelerometer bias give quadratic position error? What is aliasing and how can it masquerade as drift? Why is synchronising a 200 Hz IMU with a 1 Hz GPS harder than it looks?*

3 Sensor Time Series

Each sensor is a time series with its own units, rate, and error signature. Understanding that signature is what lets you set the fusion noise matrices $\mathbf{Q}, \mathbf{R}$ honestly (this is the seam to Phase 5 signal processing and Phase 7 fusion).

3.1 The universal sensor error model

Every real sensor output decomposes as

$$z_t = \underbrace{s_t}_{\text{true}} + \underbrace{b_t}_{\text{bias}} + \underbrace{k s_t}_{\text{scale err.}} + \underbrace{n_t}_{\text{white noise}} + \underbrace{w_t}_{\text{random walk}},$$

where bias b_t drifts slowly, white noise n_t is zero-mean and uncorrelated, and the random walk $w_t = \int n \, d\tau$ has variance growing linearly in time.

Let n_t be white with variance σ^2 . Integrating it (gyro rate $\rightarrow$ angle) gives $\text{Var}(\sum_{k \leq n} n_k) = n\sigma^2 \propto t$, so the angle error grows as $\sqrt{t}$ (**angle random walk**). Integrating a constant *bias* b gives bt (*linear*); double-integrating an accelerometer bias gives $\frac{1}{2}bt^2$ (*quadratic*). These three laws — $\sqrt{t}, t, t^2$ — are the DNA of inertial drift and the reason each sensor needs aiding.

3.2 Per-sensor summary

Sensor	Measures (units)	Rate	Dominant error	Characteristic trace
Accelerometer	specific force [a_x, a_y, a_z] (m/s ²)	100–1000 Hz	bias, vibration	$\approx g$ on one axis at rest
Gyroscope	angular vel. [$\omega_x, \omega_y, \omega_z$] (rad/s)	100–1000 Hz	bias drift , ARW	≈ 0 at rest but wandering
Magnetometer	field [m_x, m_y, m_z] (μ T)	10–100 Hz	hard/soft-iron, spikes	sinusoidal in x – y on rotation
GPS/GNSS	lat, lon, alt (deg, m)	1–10 Hz	multipath, dropouts	piecewise-smooth, jumps
Barometer	pressure $\rightarrow$ altitude (hPa, m)	10–100 Hz	slow weather drift	smooth, tracks vertical motion
IMU (MARG)	accel+gyro(+mag)	100–1000 Hz	integrated drift	see accel+gyro
Quantum mag.	scalar/vector field (nT)	10–1000 Hz	shot noise; low drift	near-drift-free field trace

3.3 Reading the signatures

Accelerometer. At rest the vector magnitude equals gravity (≈ 9.81) and points “up” in the body frame — enabling roll/pitch (not yaw) estimation. In motion, gravity and true acceleration entangle and must be separated using orientation, which is why accel-only position is hopeless without the gyro.

Gyroscope. The orientation workhorse, but bias drift is its arch-enemy: a small constant bias integrates to a linearly growing heading error that then leaks into position through the rotation matrix. Predicting this bias is a headline Trinetra-AI module (§7).

Magnetometer. Gives an absolute, *non-drifting* heading (a compass), the perfect complement to the gyro — but corrupted by **hard-iron** (additive offset) and **soft-iron** (ellipsoidal) distortion requiring calibration

$$\mathbf{m}_{\text{cal}} = \mathbf{A}^{-1}(\mathbf{m}_{\text{raw}} - \mathbf{b}).$$

Indoor magnetic *anomalies* are not just noise — they are a *map* to localise against (magnetic SLAM).

GPS / Barometer / Quantum. GPS is absolute but low-rate and absent indoors — it *defines* the GPS-denied problem. The barometer constrains the weak vertical axis. A quantum magnetometer is a future channel: cleaner field data, same pipeline.

3.4 Preprocessing checklist

1. **Timestamp & synchronise** onto a common clock/rate (interpolate slow channels).
2. **Calibrate** accel/gyro bias & scale; magnetometer hard/soft-iron.
3. **De-noise** (Phase 5: low-pass, moving average, Savitzky–Golay) without eating dynamics.
4. **Handle gaps** (mask GPS dropouts; do not silently interpolate long outages).
5. **Normalise** per channel before ML.

```
import numpy as np, pandas as pd
def resample_to(df, rate_hz, cols, t='t'):
    """Resample an irregular timestamped sensor frame onto a uniform grid."""
    grid = np.arange(df[t].iloc[0], df[t].iloc[-1], 1.0/rate_hz)
    out = {t: grid}
    for c in cols:
        out[c] = np.interp(grid, df[t].values, df[c].values)
    return pd.DataFrame(out)
```


Real datasets (see §9): OxIOD, RoNIN, MagPIE/MagPIE2, EuRoC MAV, KITTI, TUM VI, comma2k19.

4 Sliding Window

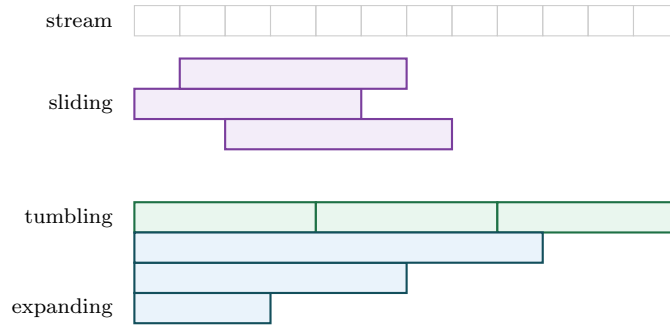
Deep models consume fixed-length *windows*, not an infinite stream. Choosing the windowing strategy is one of the highest-leverage design decisions in the project.

4.1 Formal definition

Given a stream $x_{1:T}$, a **window** of length L starting at t is the slice $x_{t:t+L-1}$. Advancing by a **stride** s produces the window set; the number of windows is

$$N = \left\lfloor \frac{T-L}{s} \right\rfloor + 1, \quad \text{overlap fraction} = 1 - \frac{s}{L}.$$

Sliding/rolling: $s < L$ (overlap). **Tumbling:** $s = L$ (no overlap). **Expanding:** start fixed, end grows (window length increases each step).



4.2 Why windows, and how the models use them

Neural nets need fixed-size inputs; a window carries the temporal context a single sample lacks (you cannot see “drift” in one sample); overlap multiplies training examples. The window length must exceed the phenomenon’s timescale — to see a ~ 1 s gait cycle at 100 Hz you need $L \gtrsim 100$.

Model	How it consumes a window
LSTM/GRU	ingest as an ordered sequence, updating a hidden state step by step; $O(L)$, streaming-friendly
TCN	dilated causal convolutions across the window in parallel; receptive field set by depth
Transformer	self-attention over all L positions at once; long-range but $O(L^2)$

```
import numpy as np
def make_windows(x, L, stride=1, horizon=1):
    """x: (T,C) -> X: (N,L,C) windows, y: (N,C) h-step-ahead targets."""
    X, y = [], []
    for s in range(0, len(x) - L - horizon + 1, stride):
        X.append(x[s:s+L]); y.append(x[s+L+horizon-1])
    return np.asarray(X), np.asarray(y)
```

```
# 1 s of 100 Hz IMU context -> next-sample target
X, y = make_windows(imu_6axis, L=100, stride=10, horizon=1)
```

Navigation examples. A 200-sample IMU window → predicted 3D displacement (the RoNIN/TLIO recipe). A window of magnetometer+position → matched against a magnetic map. A window of GPS residuals → spoofing/anomaly detector. **Design rule:** choose L from the physics (longest relevant timescale), stride from the latency budget, and **always split train/test by trajectory or time** — never by shuffled windows, or overlapping windows leak future into past and inflate scores.

5 Time Series Forecasting

Forecasting predicts future values from the past. In Trinetra-AI it is the mechanism for *anticipating drift, bridging GPS outages, and flagging faults early*.

5.1 One-step, multi-step, recursive, direct

Forecasting learns f_θ mapping a window to future values. **One-step** predicts x_{t+1} . **Multi-step (horizon h)** predicts $x_{t+1:t+h}$ by either

$$\text{recursive: } \hat{x}_{t+k} = f_\theta(\dots, \hat{x}_{t+k-1}), \quad \text{direct: } \hat{x}_{t+k} = f_\theta^{(k)}(x_{t-L+1:t}).$$

In recursive forecasting the model's own (noisy) output is fed back as input. If each step adds error ϵ and the map has local sensitivity (Lipschitz constant) λ , the k -step error grows roughly like $\sum_{j=0}^{k-1} \lambda^j \epsilon$. For $\lambda > 1$ this is *exponential* in the horizon; *direct* models avoid the feedback and hence the compounding, at the cost of training h separate heads. Encoder-decoder (seq2seq) models compromise: encode the past once, decode the whole horizon.

Teacher forcing. During training of an RNN/seq2seq decoder, feed the *ground-truth* previous value rather than the model's prediction. It speeds convergence but creates **exposure bias** (train/test mismatch), mitigated by *scheduled sampling* (gradually replace ground truth with predictions).

5.2 What Trinetra-AI can forecast — and why

Target	Forecastable?	Why it matters	Caveat
Sensor drift / bias	Yes (slow, smooth)	pre-correct before integration	must not absorb real motion
Position	short horizon	bridge brief GPS outages	diverges fast under GPS-denial
Velocity	yes (short)	smooth, gate outliers	maneuver-dependent
Orientation	short horizon	anticipate heading	yaw weakly observable w/o mag
Magnetic field	given map+position	map-matching residuals	spatial, not purely temporal
Sensor failure	yes (classification)	switch to redundant sensor	rare-event, imbalance

Key insight. *Drift is the ideal forecasting target* — smooth and low-frequency, exactly where sequence models excel — so the Drift-Prediction module forecasts next-window bias and subtracts it *before* the Kalman predict step, shrinking the effective process noise. *Raw GPS-denied position is a poor pure-forecasting target* (no absolute anchor); that is the fusion filter’s job, not a standalone forecaster.

6 Sequence Modeling

This is the heart of Phase 4: the models that turn windows into predictions.

6.1 Vocabulary

A **sequence** is ordered data $x_{1:T}$. **Sequential / temporal learning** learns a function whose output at t depends on the ordered history. A **short-term dependency** spans a few steps (gait ~ 1 s); a **long-term dependency** spans many (loop-closure over minutes). The **context window / receptive field** is how far back a model can actually see; **memory** is the internal state carrying information forward.

6.2 RNN — the baseline recurrence

A recurrent net threads a hidden state through time:

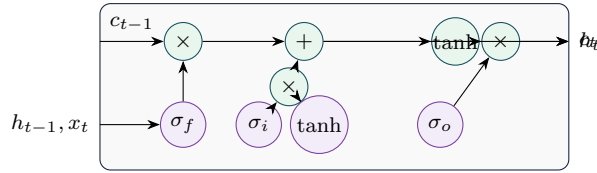
$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}), \quad \hat{\mathbf{y}}_t = \mathbf{W}_{hy}\mathbf{h}_t.$$

Back-propagation through time multiplies Jacobians across steps: $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} = \prod_{i=0}^{k-1} \mathbf{W}_{hh}^\top \text{diag}(\tanh')$. Since $|\tanh'| \leq 1$, if the largest singular value of $\mathbf{W}_{hh}$ is < 1 the product shrinks geometrically ($\rightarrow 0$, vanishing); if > 1 it explodes. Hence a plain RNN cannot learn long dependencies — motivating gating.

6.3 LSTM — gated long memory

The LSTM (Hochreiter & Schmidhuber, 1997) adds a **cell state** $\mathbf{c}_t$ and three gates:

$$\begin{aligned}
 \mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) && \text{forget} \\
 \mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) && \text{input} \\
 \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) && \text{candidate} \\
 \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t && \text{cell update} \\
 \mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) && \text{output}
 \end{aligned}$$



The additive cell update is the trick: gradients flow through $\mathbf{c}_t$ largely unimpeded, solving the vanishing-gradient problem. **Complexity** $O(T)$, sequential. **Trinetra-AI use:** primary model for drift prediction and denoising; RoNIN/TLIO use LSTM/ResNet backbones for IMU displacement.

6.4 GRU — the lean cousin

The GRU (Cho et al., 2014) merges cell and hidden state into two gates:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]), \quad \mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]), \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]).$$

Fewer parameters than LSTM, often equal accuracy, faster — a good fit for **edge deployment** (Jetson/Coral).

6.5 TCN — dilated causal convolutions

A Temporal Convolutional Network (Bai, Kolter & Koltun, 2018) stacks *causal* (no future leakage) *dilated* convolutions. With kernel size k and dilation $d_l = 2^l$ at layer l , the receptive field is

$$\text{RF} = 1 + \sum_l (k - 1) d_l,$$

growing *exponentially* with depth.

Fully parallel (fast training), stable gradients, deterministic latency — excellent for real-time drift/displacement. RoNIN ships a TCN variant; Brossard et al. (2020) denoise gyros with a dilated CNN.

6.6 Transformer, Informer, Time-Series Transformer

Self-attention (Vaswani et al., 2017) lets every position attend to every other:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

with *positional encoding* $PE_{(t,2i)} = \sin(t/10000^{2i/d})$, $PE_{(t,2i+1)} = \cos(\cdot)$ injected because attention is order-agnostic.

Captures arbitrarily long dependencies in one hop but costs $O(L^2)$ memory/time and is data-hungry. **Informer** (Zhou et al., AAAI 2021) reduces this to $O(L \log L)$ with ProbSparse attention for long-horizon forecasting; Autoformer/FEDformer/PatchTST are related efficient forecasters.

6.7 Model comparison

Model	Long-range	Parallel	Params	Complexity	Trinetra-AI fit
RNN	×	×	low	$O(T)$	baseline only
LSTM	✓	×	med	$O(T)$	drift, denoise (primary)
GRU	✓	×	low	$O(T)$	edge deployment
TCN	✓ (designed)	✓	med	$O(T)$	real-time drift/displacement
Transformer	✓✓	✓	high	$O(L^2)$	long-range multi-sensor fusion
Informer	✓✓	✓	high	$O(L \log L)$	long-horizon forecasting

Practical stance. Start with an **LSTM** or **TCN** baseline for drift/displacement (proven by RoNIN/TLIO); reach for Transformers/Informer only once you have the data volume and a genuine long-range need. A well-tuned TCN beats a poorly-trained Transformer almost every time.

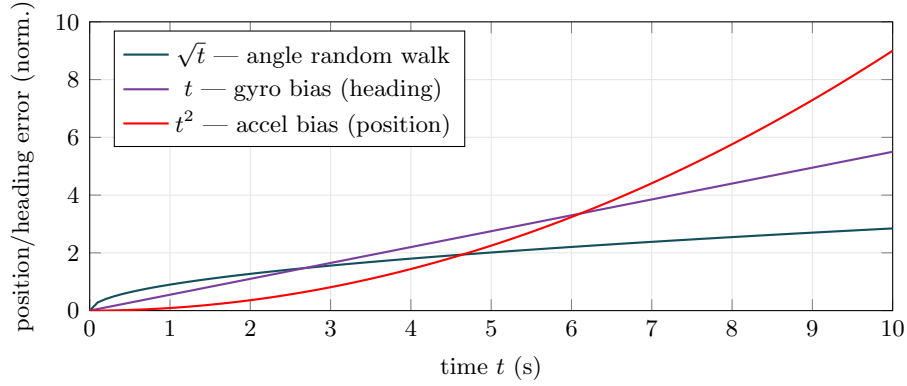
7 Drift Detection

Drift is the central enemy of GPS-denied navigation. Two distinct meanings collide here — *sensor drift* (physics) and *concept/data/model drift* (ML) — and Trinetra-AI must handle both.

7.1 Sensor drift (physical)

Type	Cause	Effect on navigation
Bias drift	slow change in zero-offset	integrates into growing error
Temperature drift	bias/scale vary with temperature	correlated, predictable from temp channel
Gyroscope drift	bias + angle random walk	heading error $\propto t$, leaks to position
Accelerometer drift	bias	position error $\propto t^2$
Magnetic drift	environmental field change; hard/soft-iron shift	heading reference corrupted

Gyro heading error from bias b_g : $\Delta\psi(t) = \int_0^t b_g d\tau = b_g t$. Accelerometer position error from bias b_a : $\Delta p(t) = \frac{1}{2} b_a t^2$. Angle random walk from white gyro noise: $\sigma_\psi(t) \propto \sqrt{t}$. Even a partial bias estimate removes the dominant, fastest-growing term — which is exactly why drift *prediction* pays off so much.



7.2 Concept / data / model drift (ML)

Data drift: the input distribution shifts (new terrain, vehicle, temperature). **Concept drift:** the input→output relationship itself changes (sensor ages, needs re-calibration). **Model drift:** a deployed model degrades as the world moves away from its training distribution.

Detecting these tells Trinetra-AI when to retrain or fall back to the classical filter — a health-monitoring concern.

7.3 Detection & prediction methods

Family	Methods	Role in Trinetra-AI
Statistical	CUSUM, Page-Hinkley, ADWIN, KS-test	cheap online change alarms on residuals
Traditional	high-pass filtering, regression on temperature	remove predictable thermal/bias trends
Kalman-based	augment state with bias; innovation monitoring	filter estimates bias online
Machine learning	one-class SVM, isolation forest	fault/anomaly flags
Deep learning	LSTM/TCN bias forecast; autoencoder residual	the Drift-Prediction & anomaly modules
Adaptive	LMS/RLS; adaptive Q/R	retune fusion as conditions change

The cumulative-sum statistic accumulates deviations of the residual r_t from its expected mean μ_0 :

$$S_t = \max(0, S_{t-1} + (r_t - \mu_0 - \kappa)), \quad \text{alarm if } S_t > h,$$

with slack κ and threshold h . Applied to the Kalman *innovation*, it raises a drift alarm the

moment residuals start trending — feeding the adaptive-weighting module.

The classical answer is elegant: *augment the state with the biases* and let the Kalman filter estimate them, using the **innovation** (measurement – prediction) as the observable drift signature. Trinetra-AI’s contribution is to *predict* drift ahead of time with a learned model and pre-empt it, rather than only reacting through innovations.

8 Temporal Feature Engineering

Even with deep models, hand-built temporal features remain invaluable — for classical ML modules (fault/health classifiers), interpretability, and as auxiliary NN inputs. Features are computed *per window* (§4).

8.1 Feature catalogue

Feature	Definition	Captures
Lag	x_{t-k}	raw short memory
Rolling mean	$\frac{1}{L} \sum x$	local level / trend
Rolling std	window std	volatility (vibration, drift on-set)
Rolling min/max	window extremes	spikes, saturation
Autocorrelation	$\rho(k)$	periodicity, smoothness
Cross-correlation	$\rho_{xy}(k)$ between channels	lead-lag, time-sync
FFT features	dominant freq, band power	vibration/gait signature
Wavelet features	multi-scale coeffs	transient + frequency localisation
Peak detection	count/height of peaks	steps, impacts, events
Statistical	skew, kurtosis, RMS, energy, ZCR	distribution / signal shape

Two workhorse shape features:

$$\text{RMS} = \sqrt{\frac{1}{L} \sum_i x_i^2}, \quad \text{ZCR} = \frac{1}{L} \sum_i \mathbb{I}[x_i x_{i-1} < 0].$$

The FFT/wavelet features are the bridge to **Phase 5**, where the same transforms appear as filtering tools.

8.2 Which features for which module

Module	Most useful features
Drift prediction	rolling mean/std, low-freq FFT band, temperature cross-corr, lags
Fault detection	rolling std spikes, max/min, ZCR, reconstruction error
Anomaly detection	Mahalanobis on features, autoencoder residual, forecast residual
Sensor health	SNR estimate, rolling-std trend, autocorrelation decay
Adaptive weighting	per-sensor variance, innovation magnitude, confidence score
Trajectory prediction	velocity/heading lags, rolling mean, gait FFT peak

Given feature vector $\mathbf{f}$ with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$, the squared Mahalanobis distance $D^2 = (\mathbf{f} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{f} - \boldsymbol{\mu})$ measures deviation in units of standard deviations, accounting for correlations. A measurement with D^2 above a χ^2 threshold is flagged as a fault — the same test gates outliers inside the Kalman filter.

```
import numpy as np
def window_features(w): # w: (L,) single-channel window
    fft = np.abs(np.fft.rfft(w))
    return {"mean": w.mean(), "std": w.std(), "min": w.min(), "max": w.max(),
            "rms": np.sqrt((w**2).mean()),
            "zcr": np.mean(w[1:]*w[:-1] < 0),
            "dom_freq_bin": int(fft.argmax()),
            "band_energy_low": fft[:len(fft)//4].sum()}
```

Deep models learn features end-to-end, but engineered features give a **strong, interpretable classical baseline** (Phase-2 XGBoost/RandomForest) for the health/fault modules — a baseline you can trust when the neural net is uncertain.

9 Time Series Datasets

All entries are real and verified; confirm the current license/download on the official page before use. Difficulty is relative to a solo research project.

Dataset	Sensors	Ground truth	Diff.	Notes
OxIOD	IMU (accel/gyro/-mag)	Vicon/VI	Easy–Med	large, labeled; best first inertial-DL set (Chen et al.)
RoNIN	IMU (phone)	2-device traj.	3D Med	~42.7 h, >100 subjects; ATE/RTE benchmark (Herath et al., ICRA 2020)
TLIO	IMU (headset)	VIO	Med	pairs with tight EKF fusion (Liu et al., RAL 2020)
MagPIE MagPIE2	/ IMU + magnetometer	cm-level	Med–Hard	magnetic anomalies, multi-height (Hanley et al., IPIN 2017; MagPIE2 IEEE)
EuRoC MAV	IMU + stereo	Vicon/Leica	Med	standard VIO benchmark (Burri et al., 2016)
KITTI	IMU+GPS/RTK+LiDAR+RTK+GPS/INS	RTK+GPS	Med	driving odometry; no magnetometer (Geiger et al.)
TUM VI	IMU + stereo	partial MoCap	Med	precise IMU-cam sync (Schubert et al., 2018)
comma2k19	IMU+GNSS+CAN	GNSS+fusion	Med	33 h highway; GNSS+IMU fusion
KAIST Urban	IMU+GPS+LiDAR	fused	Hard	urban-canyon multi-path
UCI HAR	phone IMU (no mag)	activity labels	Easy	ML pipeline / windowing practice

Synthetic option. Where no magnetometer track exists (e.g. KITTI), simulate one by sampling the *World Magnetic Model* along the ground-truth path with calibrated noise. Gazebo/Air-Sim can generate full IMU+mag+baro streams for controlled GPS-denial ablations with perfect ground truth.

Recommended starting pair: RoNIN or OxIOD (inertial displacement) + **MagPIE2** (magnetic map-matching) — together they cover both halves of Trinetra-AI’s story with real ground truth.

10 Libraries

Library	Key functions	Trinetra-AI use	Limitation
NumPy	fft, broadcasting, linalg	windowing, feature math	low-level only
SciPy	signal, interpolate, welch	de-noise, resample, PSD	not ML
pandas	rolling, resample, interpolate	sync, rolling features, IO	slow at very high rate
statsmodels	adfuller, ARIMA, acf	stationarity, baselines	weak for multivariate DL
tsfresh	extract_features	bulk features for health/fault ML	huge feature count; prune
sktime	unified TS ML API	classical baselines, TS-CV	learning curve
darts	RNNModel, TCNModel, TFTModel	quick DL forecasting baselines	abstraction hides detail
gluonts	DeepAR, prob. heads	uncertainty forecasts	backend friction
pytorch-forecasting	TFT, TimeSeriesDataSet	production drift models	heavier setup
PyTorch (core)	nn.LSTM/GRU/Conv1d/Transformers	all custom models	you build plumbing

Stack: numpy/scipy/pandas for the data layer, statsmodels/tsfresh for classical baselines/features, and **PyTorch** for the custom drift/denoise/fusion networks. Prototype a forecasting baseline with darts or pytorch-forecasting before committing to a bespoke model.

11 Research Papers

Verified, correctly attributed. Priority: ★★★ read first, ★★ core, ★ as needed.

11.1 Foundational sequence models

Paper	Authors, year	Contribution	Pri.
Long Short-Term Memory	Hochreiter & Schmidhuber, 1997	gated memory, no vanishing grad	***
Learning Phrase Representations (GRU)	Cho et al., 2014	simpler gated recurrence	**
Attention Is All You Need	Vaswani et al., 2017	the Transformer	***
Generic Convolutional ... (TCN)	Bai, Kolter & Koltun, 2018	dilated causal convs rival RNNs	***
Informer	Zhou et al., AAAI 2021	$O(L \log L)$ long-seq forecasting	**

11.2 Learned inertial navigation (most relevant)

Paper	Authors, year	How it helps Trinetra-AI	Pri.
IONet	Chen, Lu, Markham & Trigoni, AAAI 2018	first DNN for inertial odometry (“cure the curse of drift”)	***
RoNIN	Herath, Yan & Furukawa, ICRA 2020	robust neural inertial nav; ResNet/LSTM/TCN; ATE/RTE + dataset	***
TLIO	Liu et al., RA-L 2020	learned 3D displacement + uncertainty <i>tightly fused in an EKF</i>	***
OxIOD (dataset)	Chen et al., 2018	large labeled inertial-odometry data	**
Denoising IMU Gyroscopes w/ DL	Brossard, Bonnabel & Barrau, RA-L 2020	dilated-CNN gyro de-noising (Phase-5 bridge)	**

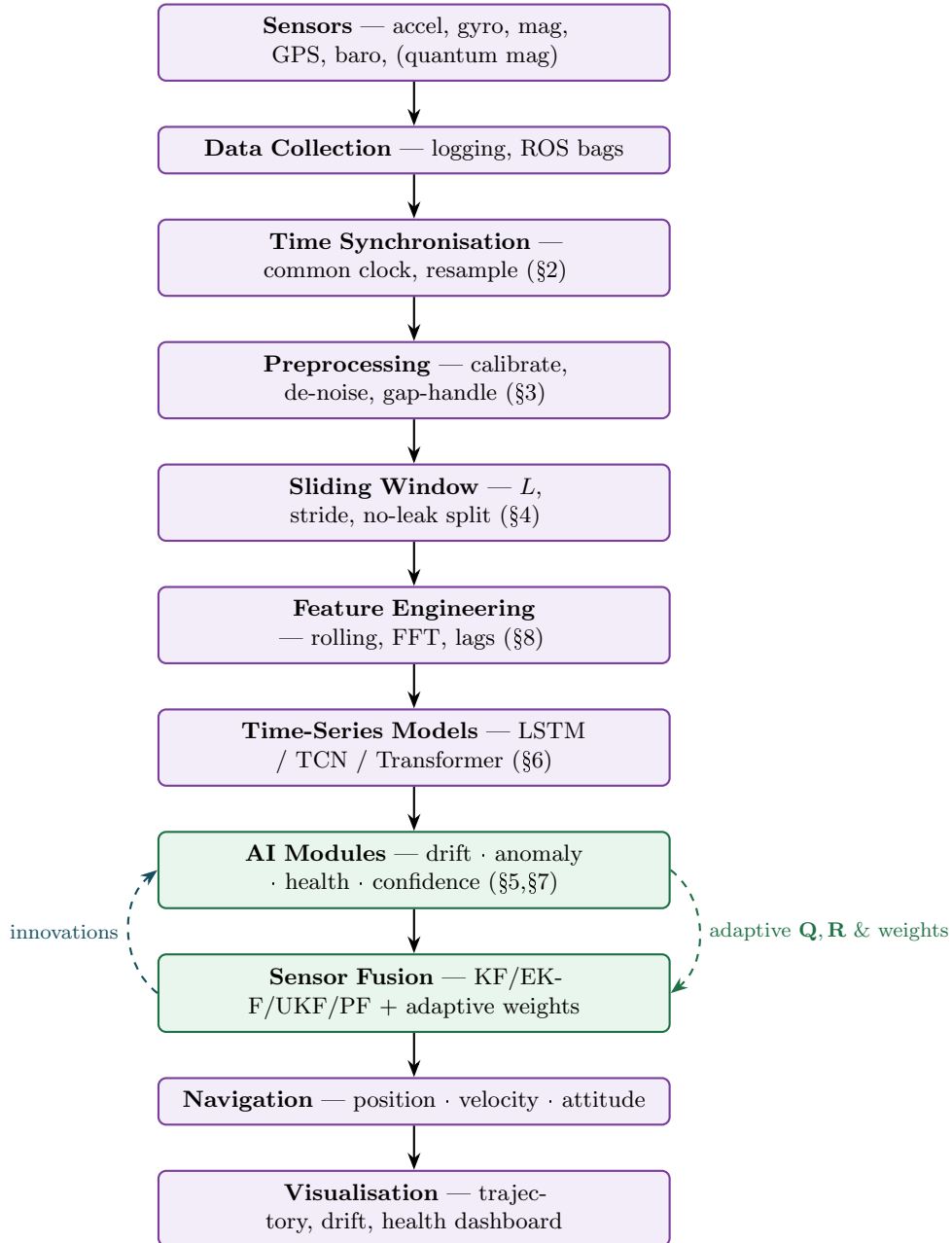
11.3 Magnetic & generative (optional)

Paper	Authors, year	Contribution	Pri.
MagPIE / MagPIE2	Hanley et al., IPIN 2017 / IEEE	magnetic-anomaly positioning data	**
TimeGAN	Yoon, Jarrett & van der Schaar, NeurIPS 2019	realistic TS synthesis	*
Auto-Encoding Variational Bayes (VAE)	Kingma & Welling, 2013	latent generative / denoising	*

Reading path: IONet → RoNIN → TLIO gives the entire “learned inertial + Kalman fusion” arc that Trinetra-AI extends; read the LSTM/Transformer/TCN foundations alongside. **Citation hygiene:** verify every DOI/venue on the official page before it enters a report — fabricated references are the fastest credibility loss with the reviewers you are targeting.

12 Project Integration

Here is where every time-series concept lands in the Trinetra-AI pipeline. The dashed loops are the intelligence: AI modules tune the filter forward, and the filter’s innovations inform the AI modules backward.



Block detail. *Synchronisation* puts multi-rate sensors on one timeline (the make-or-break step); *preprocessing* calibrates and de-noises; *windowing* segments the streams (split by trajectory to avoid leakage); *feature engineering* feeds classical modules and augments the networks; *time-series models* produce displacement, bias forecasts, denoised signals, and anomaly scores; the *AI modules* feed adaptive $\mathbf{Q}$, $\mathbf{R}$ and per-sensor weights into the *fusion* core, which returns innovations that drive drift/anomaly detection — a closed loop. *Navigation* outputs the fused state plus a health/confidence readout usable in GPS-denied mode.

13 Interview Preparation

13.1 30 interview questions

1. Define a time series and why order matters.
2. Why does accel bias give quadratic position error?

3. Sampling frequency, Nyquist, aliasing.
4. How aliasing masquerades as drift.
5. Time-sync of 200 Hz IMU with 1 Hz GPS.
6. Strict vs weak stationarity.
7. How to test stationarity.
8. Trend vs seasonality vs cyclic (sensor examples).
9. Autocorrelation on sensor data.
10. Hard- vs soft-iron distortion & correction.
11. Why gyro bias drift dominates orientation error.
12. The five window types.
13. Choosing window length and stride.
14. Why split by trajectory not randomly.
15. One-step vs multi-step; recursive vs direct.
16. Teacher forcing & exposure bias.
17. Why drift is a good target but GPS-denied position a poor one.
18. LSTM gates & vanishing gradients.
19. GRU vs LSTM.
20. Dilated causal conv & receptive field.
21. Self-attention complexity; positional encoding.
22. What Informer solves.
23. Sensor vs concept/data/model drift.
24. Three change detectors.
25. KF online bias estimation.
26. The innovation as a drift/anomaly signal.
27. Features for fault vs drift.
28. Detecting a spoofed GPS stream.
29. How AI modules adapt **Q/R**.
30. Metrics for a GPS-denied system.

13.2 20 research questions

1. Can a learned drift predictor beat innovation-based bias estimation in long outages?
2. Window length vs accuracy/latency trade-off for displacement.
3. Cross-sensor attention vs independent models.
4. Transfer from pedestrian to vehicle/UAV.
5. Do TLIO-style uncertainties improve adaptive weighting?
6. Drift-reduction ceiling from a simulated quantum magnetometer.
7. Robustness of mag map-matching to “live-load” changes.
8. TCN vs Transformer at equal latency.
9. Leakage-free splitting for overlapping windows.

10. Teacher-forcing vs scheduled sampling on long horizons.
11. Best change detector for ML→classical fallback.
12. TimeGAN augmentation for rare maneuvers.
13. Temperature-conditioned vs agnostic drift models.
14. Minimal feature set for sensor-health.
15. Fusion-loop stability when ML is wrong.
16. Quantifying sync-error impact on fusion.
17. Direct vs recursive for outage bridging.
18. Cross-dataset generalisation.
19. Calibrated confidence for safety.
20. Ablations isolating each AI module.

13.3 10 viva questions

1. Derive position-error growth from accel and gyro biases.
2. Where does time series appear across your pipeline?
3. Justify window length from the physics.
4. Why is the Kalman filter a time-series (recursive Bayesian) method?
5. Defend LSTM/TCN over a Transformer for your first model.
6. How do you prevent leakage in evaluation?
7. What does your drift module output and how is it consumed?
8. How do you validate a claimed improvement (baselines, ablations, metrics)?
9. Failure modes of learned components and your safeguards.
10. How would a quantum magnetometer change the design without changing the pipeline?

14 Practical Implementation

Notebook	Contents	Expected output
01.data_sync.ipynb	load RoNIN/OxIOD/MagPIE, resample, sync	aligned multi-sensor arrays
02.eda_stationarity.ipynb	plots, ADF, decomposition, ACF	signal characterisation
03.calibration_denoise.ipynb	hard/soft-iron, filters	calibrated, de-noised streams
04.windowing_features.ipynb	windowing, tsfresh/manual features	(N,L,C) tensors + table
05.drift_lstm_tcn.ipynb	train LSTM & TCN drift models	models, loss curves
06.anomaly_health.ipynb	autoencoder + classifier	anomaly scores, PR curves
07.fusion_integration.ipynb	EKF/UKF + AI corrections	fused trajectory
08.eval_dashboard.ipynb	ATE/RTE, drift-rate, ablations	results tables + dashboard

Evaluation metrics. ATE (absolute trajectory error) and RTE (relative), position RMSE (m), orientation error (deg), *drift rate* (error per distance/time — the honest GPS-denied metric), forecast error vs horizon, anomaly PR-AUC, health calibration, and compute latency on target hardware.

Visualisation. Trajectory-vs-ground-truth (2D/3D), drift-over-time, per-sensor health strip, error-vs-horizon curves, covariance ellipses, magnetic-map overlays (Matplotlib/Plotly; Streamlit/Dash for the live view).

Common mistakes. Random-shuffling overlapping windows (leakage); ignoring time sync; over-filtering (eating dynamics); interpolating across long GPS gaps (hiding the problem); *no* classical baseline (strawman comparison); fabricated citations.

Best practices. Version data & configs; fix seeds; log experiments; unit-test the windowing/sync code; always compare against a *well-tuned* classical filter; run ablations removing one AI module at a time; keep and calibrate uncertainty outputs.

15 Summary, Glossary & Roadmap

15.1 Summary

Time series is the substrate of Trinetra-AI. Sensor streams are ordered, non-stationary, and drift-prone; windows turn them into model inputs; sequence models (LSTM/TCN/Transformer) forecast drift, denoise, and score anomalies; and these feed a classical Kalman fusion core in a closed loop that adapts **Q**, **R** forward and consumes innovations backward. The dominant enemy — unbounded drift under GPS-denial — is precisely a temporal, forecastable phenomenon, which is why this phase is foundational rather than peripheral.

15.2 Glossary

ARW — angle random walk; gyro noise whose angle error grows as $\sqrt{t}$. **ATE / RTE** — absolute / relative trajectory error. **Autocorrelation** — correlation of a signal with a lagged copy of itself. **Bias** — slowly-varying sensor offset; integrates into drift. **Dead reckoning** — estimating position by integrating motion from a known start. **Drift** — error growth over time from integrating biased/noisy sensors. **Hard/soft-iron** — additive / ellipsoidal magnetometer distortions. **Innovation** — measurement minus prediction inside a Kalman filter. **Nyquist frequency** — $f_s/2$; highest representable frequency. **Receptive field** — span of input a model position can see. **Stationarity** — time-invariance of a signal’s statistics. **Teacher forcing** — training a decoder on ground-truth previous outputs. **Window (L , stride)** — fixed-length model input; step between windows.

15.3 Roadmap: how Phase 4 connects to Phase 5 (Signal Processing)

This phase repeatedly reached for tools that Phase 5 formalises.

Phase 4 touchpoint	Becomes in Phase 5
Noise, de-noising in preprocessing	low/high/band-pass, Butterworth, moving average, Savitzky–Golay
FFT features (§8)	Fourier analysis, spectra, PSD (Welch), leakage
Aliasing / Nyquist (§2)	sampling theory, anti-alias filtering
Wavelet features (§8)	multi-resolution / wavelet transforms
Sensor noise model (§3)	Allan variance, noise-density characterisation for Q , R
Random walk / autocorrelation	stochastic-process view of sensor noise

Enter Phase 5 with a working mental model of *why* you filter — to expose the true signal and characterise noise for fusion — and DSP becomes the toolbox that makes the time-series and fusion layers trustworthy. (Phases 2 & 3 — ML and Deep Learning — are assumed complete; this document leans on that background where sequence models appear.)

End of Phase 4 notes. Next: Phase 5 (Signal Processing) → Phase 6 (Sensors & IMU) → Phase 7 (Sensor Fusion & Kalman Filters).